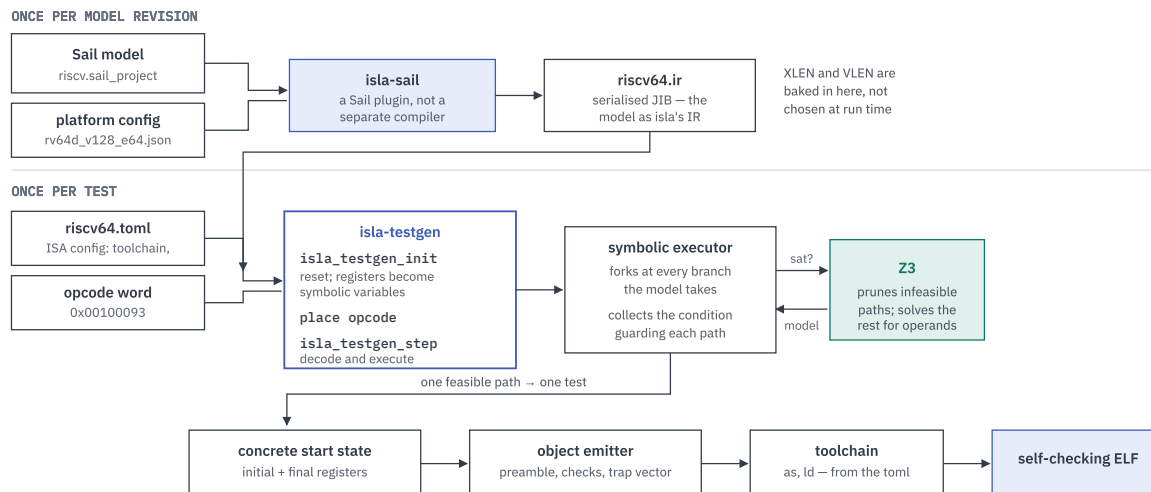


Inside isla-testgen

How one opcode becomes a self-checking ELF: the Sail model is compiled to an IR, executed symbolically with unknown registers, and the solver is asked for values that reach each feasible path.

The whole flow



The solver is never handed a complete constraint system. isla drives Z3 incrementally, asking after each branch whether the path so far is still satisfiable, so an infeasible path is abandoned as soon as it becomes one.

Two rates: the IR is rebuilt only when the model or the platform config changes; everything below the divider runs per test.

Stage by stage

1 • Compile the model to IR

isla-sail is a *Sail plugin* — it registers itself as a Sail target and links against **libsail**, rather than being a separate compiler. It emits the model as serialised **JIB**, Sail's own in-memory IR, into a **.ir** file.

```
isla-sail --isla-preserve isla_testgen_init --isla-preserve isla_testgen_step \
--config build/config/rv64d_v128_e64.json --all-modules riscv.sail_project \
-o riscv-ir/riscv64
```

--isla-preserve is not optional. **isla-sail** prunes anything unreachable from a small hardcoded root list, and would silently drop the two entry points as dead code. The build then succeeds and the generator fails at run time with a missing function — which is why a post-build check confirms they survived.

XLEN is a run-time selector; VLEN is not. The platform config is consumed *here*, so `type vlenbits = bits(vlen)` monomorphises into the IR — a vector register becomes a fixed-width type. A configuration whose VLEN does not match the available IR needs a rebuild; it cannot be selected later. This is why VLEN 256 and 512 route to the oracle instead.

2 · Load the IR and the ISA config

The `.toml` is a different config from the model's platform JSON, and easy to confuse with it. It tells isla about the *environment* rather than the architecture:

KEY	WHAT IT SUPPLIES
<code>pc</code>	Which register is the program counter
<code>assembler</code> , <code>linker</code> <code>objdump</code> , <code>nm</code>	The toolchain, with its <code>-march</code> string. Resolved on <code>PATH</code> when not absolute
<code>[threads]</code>	Where generated code is placed
<code>[symbolic_addrs]</code>	The region symbolic addresses are solved into
<code>[registers.defaults]</code>	Reset values the model would otherwise leave unconstrained

3 · Build a symbolic start state

The generator calls the model's own `isla_testgen_init`. This runs the model's reset, then leaves the registers under test as **symbolic variables** rather than numbers: `x10` is not 0, it is an unknown the solver may later choose.

This is the two-phase convention the model does not otherwise expose, and the reason Deliverable 5 exists.

4 · Place the opcode and step

The opcode word is written into memory at the code address, and `isla_testgen_step` is called. The model decodes and executes it — the same code path a real fetch would take.

5 · Fork, and ask the solver

Wherever the model branches on something symbolic, the executor **forks**: it continues down both sides, accumulating the condition that guards each. After each fork it asks Z3 whether the path so far is still satisfiable, and abandons the path as soon as it is not.

This is what “aiming” means. The path condition for the divide-by-zero case is `x11 == 0`; for signed overflow it is `x10 == INT_MIN && x11 == -1`. The solver is asked to satisfy that condition, so the operands are *derived from the behaviour* rather than chosen and checked.

6 · Read back a concrete state

For each surviving path, Z3 returns a model — concrete values for every symbolic variable. Those become the test's initial register values, and the register state the model reached becomes the expected final values.

7 · Emit and materialise

The object emitter writes assembly: the preamble that installs the trap vector and loads the initial values, the opcode under test, the comparison against the expected final state, the trap vector itself, and the HTIF `tohost` write that reports the verdict. The toolchain named in the `.toml` assembles and links it.

```
isla-testgen -A riscv-ir/riscv64.ir -C riscv-ir/riscv64.toml -a riscv64 \  
--memory-region 0x80020000-0x80030000 \  

```

```
-o /tmp/out -n 1 0x00100093      # addi x1, x0, 1
```

`--test-file` instead of an ELF gives a plain-text dump of the solved initial and final state — no toolchain needed, and the fastest way to check that a generated test means what you think it means.

Where it stops

LIMIT	CAUSE	CONSEQUENCE
129-bit values	isla's concrete bitvector representation is bounded	VLEN 256 and 512 cannot be represented at all
Vector state	Vector length is run-time state the symbolic path cannot fix	Anything reaching <code>read_vreg</code> fails outright
Floating point	The model calls SoftFloat; the engine has no binding for it	FP terminates with a missing function
Compressed control flow	The engine does not model a compressed instruction's control-flow effect	<code>c.jr</code> / <code>c.jalr</code> emerge with the jump target unconstrained
Cost	Solver time per test, memory-bound before time-bound	Seconds per test against milliseconds for the oracle

Each of these is why a target routes elsewhere. They are representational limits recorded with the routing decision, not defects to be fixed.

Commands and file names taken from the isla-gen-extension README and `riscv-ir/riscv64.toml`; entry points from `src/target.rs`.